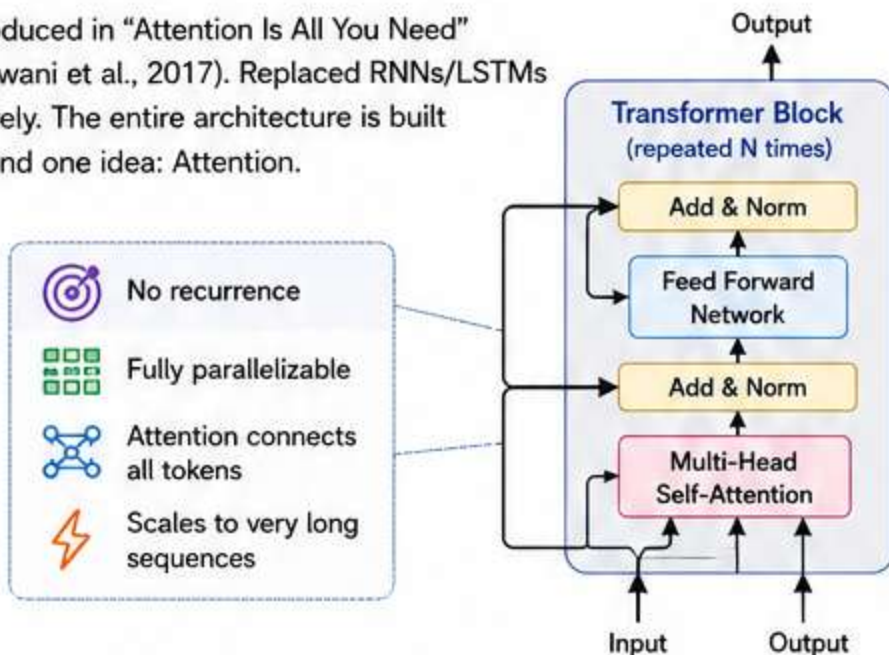


TRANSFORMER ARCHITECTURE

1. The Transformer — The Engine Behind Every LLM

Introduced in “Attention Is All You Need” (Vaswani et al., 2017). Replaced RNNs/LSTMs entirely. The entire architecture is built around one idea: Attention.

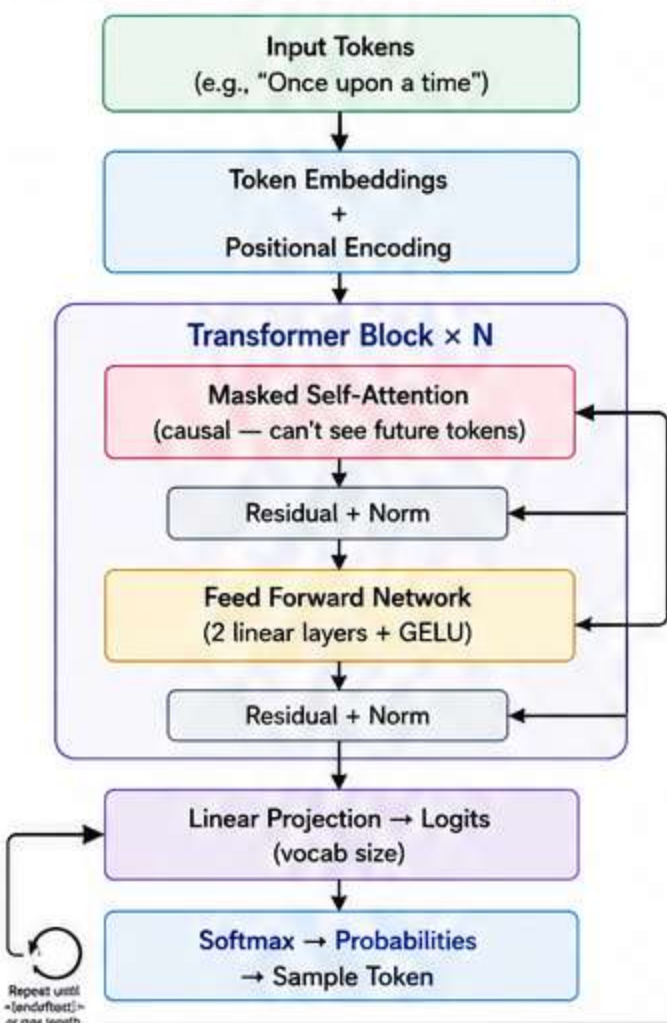


2. Encoder vs Decoder vs Encoder-Decoder

Type	Examples	Best For
Encoder Only	BERT, RoBERTa	Classification, embeddings, understanding
Decoder Only	GPT, LLaMA, Mistral	Text generation, chat, completion
Encoder-Decoder	T5, BART, mT5	Translation, summarization, seq2seq

★ Modern LLMs are almost all Decoder-Only.

3. Decoder Architecture — Deep Dive



4. Self-Attention — The Core Mechanism

Every token asks: “Which other tokens should I pay attention to?”

Formula:

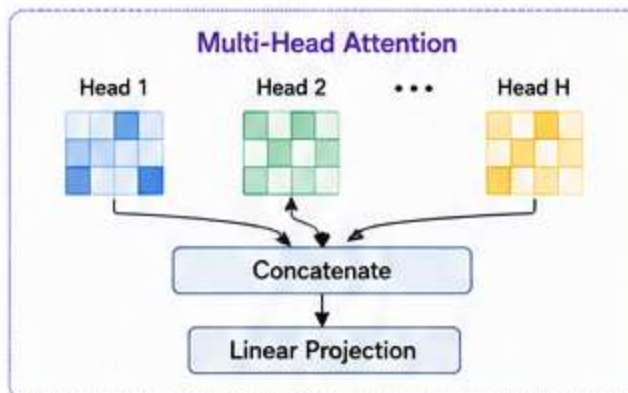
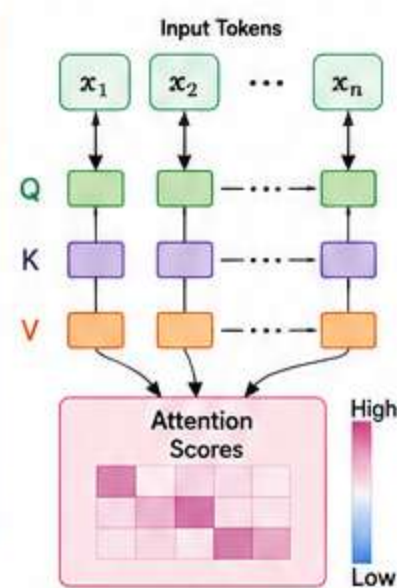
$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) \cdot V$$

■ Q (Query) — What am I looking for?

■ K (Key) — What do I have to offer?

■ V (Value) — What information do I carry?

■ $\sqrt{d_k}$ — Scaling factor to prevent vanishing gradients in softmax



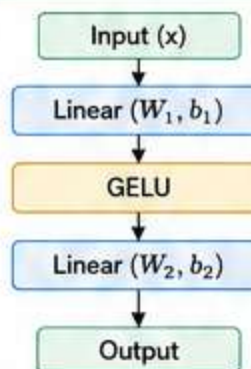
- Run attention H times in parallel with different learned projections
- Each head learns different relationships (syntax, coreference, semantics)
- Outputs are concatenated and projected

5. Positional Encoding Types

Type	Used In	Notes
Sinusoidal (Absolute)	Original Transformer	Fixed, not learned
Learned Absolute	GPT-2, BERT	Position embedding table
RoPE (Rotary)	LLaMA, Mistral, GPT-NeoX	Relative, extends well to long context
ALiBi	MPT, BLOOM	Adds linear bias to attention scores
NoPE	Some modern models	No positional encoding at all

6. Feed Forward Network (FFN)

$$\text{FFN}(x) = \text{GELU}(xW_1 + b_1)W_2 + b_2$$



Typically 4× the hidden dimension (e.g., hidden=4096 → FFN=16384)

Acts as a “memory” — stores factual associations

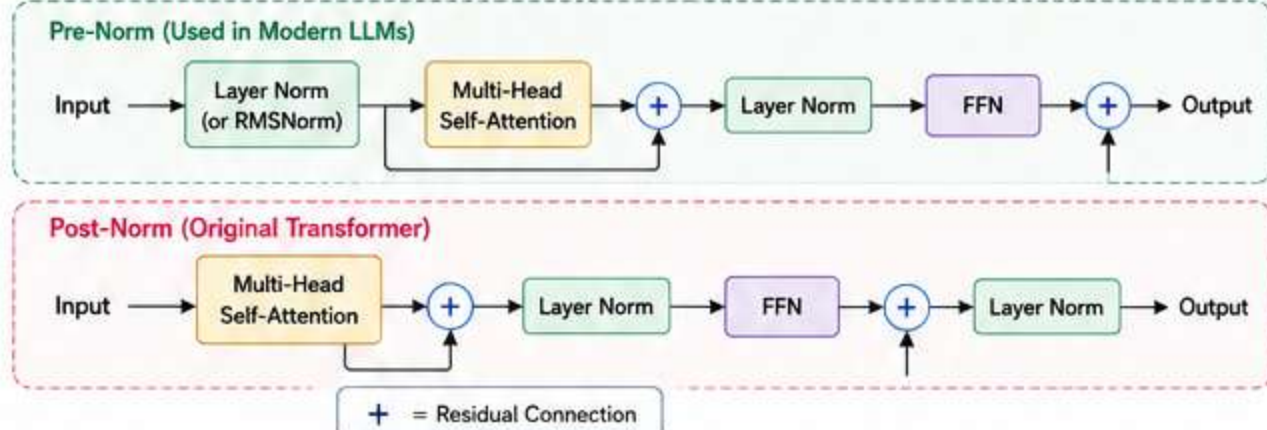
SwiGLU activation used in LLaMA, PaLM for better performance

7. Layer Normalization

Applied before attention (Pre-Norm) in modern LLMs — more stable than Post-Norm

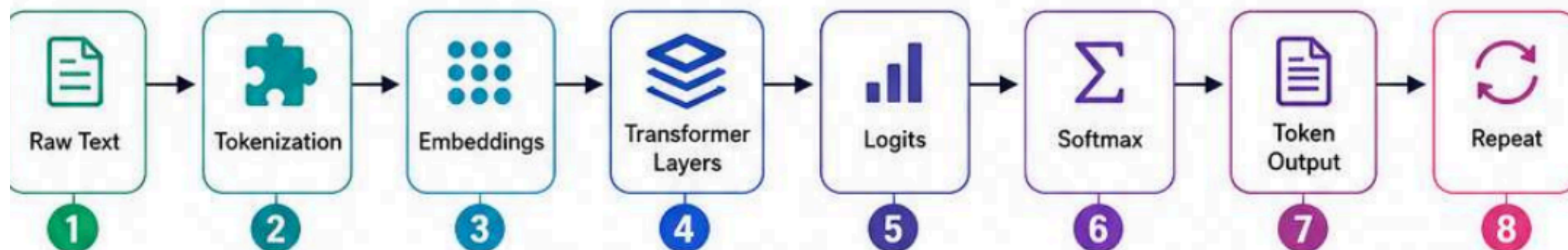
RMSNorm (Root Mean Square Norm) used in LLaMA — simpler and faster than LayerNorm

Keeps activations in a stable range throughout deep networks



How LLMs work: the Transformer generation pipeline

2. How LLMs Work — Behind the Scenes



- 1 Tokenization** — Text is split into tokens (words, subwords, characters). “ChatGPT” → [“Chat”, “G”, “PT”]. Vocab size is typically 32K–100K tokens.
- 2 Embedding** — Each token is converted into a high-dimensional vector (e.g., 768 or 4096 dimensions) that encodes meaning.
- 3 Positional Encoding** — Since transformers have no built-in sense of order, position info is injected into embeddings.
- 4 Transformer Layers** — The core engine. Each layer applies Self-Attention + Feed Forward Network + Layer Norm.
- 5 Logits & Softmax** — The final layer outputs a score (logit) for every token in the vocabulary. Softmax converts them to probabilities.
- 6 Sampling / Decoding** — A token is selected based on the probability distribution. Repeat until end token or max length.

This diagram mainly describes text generation by a decoder-only Transformer.

Connecting the image to Transformers

Example prompt: "The capital of India is". A decoder-only model processes this prompt and predicts the next token. It then adds that token to the text and continues.

1. Raw text and tokenization

The tokenizer splits the prompt into token IDs. Tokens can be words, parts of words, or characters. The exact split depends on the tokenizer; the image's ChatGPT split is only illustrative.

2. Embeddings and position information

Each token ID is mapped to a learned vector of numbers. Position information helps the model distinguish word order. Some models add position embeddings; others use methods such as RoPE within attention.

3. Transformer layers: where context is learned

Masked self-attention lets each token use its own and earlier tokens' information. It cannot look at future tokens. A feed-forward network transforms each token's representation. Residual connections and normalization help information flow through many layers. These blocks are repeated.

4. Logits: scores for the next token

A final vocabulary projection converts the last position's representation into one score for every possible next token. These scores are called logits; they are not yet probabilities.

5. Softmax and token selection

Softmax converts the scores into probabilities. A decoding rule selects the next token, for example by choosing the highest probability or sampling. The model might select " New", then " Delhi"; the actual token split varies.

6. Repeat to produce a response

The selected token becomes part of the input for the next prediction. Generation continues until a stopping condition, such as an end token or length limit. Training can process many positions in parallel, while this generation loop proceeds one token at a time.

How the three architectures differ

Encoder-only: reads the input with bidirectional attention and produces contextual representations, often for classification or embeddings. It usually does not use this repeating text-generation loop.

Decoder-only: follows the generation loop in the image, using causal attention.

Encoder-decoder: an encoder processes the source input; a decoder generates the output, using causal self-attention and cross-attention to the encoder output.